

# Défense 4DOP1DR — les 13 questions

A/B testing Spring Boot / Django sur Azure · matricule 66045 · fiche de révision

## 1. Expliquez une instruction précise dans l'un de vos Dockerfiles, du gitlab-ci.yml ou de votre script Terraform.

Méthode 3 temps : **ce que c'est, ce que ça fait, pourquoi**. Ex. `COPY --from=build /app/target/*.jar app.jar` : « C'est la copie du multi-stage : je prends le jar produit par Maven dans le stage `build` et je le mets dans l'image finale légère (JRE). `--from=build` = depuis la première image. »

## 2. Commentez/supprimez l'instruction EXPOSE. Reconstituez l'image, redémarrez. L'app fonctionne-t-elle toujours ?

**Oui**. Je commente `EXPOSE`, `docker compose up --build`, je teste → ça marche. `EXPOSE` est **documentaire** ; le vrai mapping de port c'est `ports: "8080:8080"` dans le compose. Donc `EXPOSE` ne change rien au fonctionnement.

## 3. Modifiez le port de l'app Django (ex. 8000 → 8050). Quels ajustements ?

**3 fichiers** : `django/Dockerfile` (`EXPOSE 8050` + `runserver 0.0.0.0:8050`), `docker-compose.yml` (`ports: "8050:8050"`), `nginx/nginx.conf` (`server django:8050;`). Si j'en oublie un, ça casse.

## 4. Ajoutez/supprimez depends\_on. Observez l'ordre de démarrage. Impact ? Pourquoi/quand l'utiliser ?

`depends_on` garantit l'**ordre** de démarrage (nginx après spring/django). Si je l'enlève, nginx peut démarrer avant les apps → premières requêtes en **502**. MAIS ça ne garantit **pas** que l'app soit **prête** à répondre, juste que son conteneur est lancé → pour la vraie disponibilité il faut un `healthcheck` (TD 04). Utile dès qu'un service dépend d'un autre.

## 5. Ajoutez l'analyse SonarQube au pipeline. Expliquez les étapes, montrez les résultats, l'impact.

Pas mis dans mon projet (c'est le TD 06, séparé de mon scénario). Je peux écrire le stage ; l'analyse complète nécessite un **serveur SonarQube lancé + un token**. SonarQube remonte **bugs, code smells, failles, couverture de tests**.

```
sonarqube:
  stage: sonarqube
  image: sonarsource/sonar-scanner-cli:latest
  script:
    - sonar-scanner -Dsonar.host.url=$SONAR_HOST_URL
      -Dsonar.login=$SONAR_TOKEN -Dsonar.projectKey=projet-66045 -Dsonar.sources=.
```

## 6. Ajoutez une variable DEPLOY\_ENV et utilisez-la dans une condition (job qui ne tourne que si = production).

Une variable globale + une règle sur le job. Le job ne tourne que si la condition matche. La variable peut venir du yaml, des Settings GitLab, ou d'un lancement manuel. (TD 07 : variables p.6, rules p.16)

```
variables:
  DEPLOY_ENV: "staging"
deploy-prod:
  stage: deploy
  script: - echo "deploy prod"
  rules:
    - if: '$DEPLOY_ENV == "production"'
```

## 7. Modifiez le pipeline pour qu'il ne se déclenche que sur un tag spécifique (ex. presentation-2026). Expliquez le déclenchement conditionnel.

Au niveau global : `workflow.rules` avec `$CI_COMMIT_TAG`. Puis `git tag presentation-2026 && git push origin presentation-2026` → le pipeline tourne ; un push normal → rien. `$CI_COMMIT_TAG` est injecté par GitLab sur un push de tag. (TD 07 p.16)

```
workflow:
  rules:
    - if: '$CI_COMMIT_TAG == "presentation-2026"'
```

## 8. Vous avez buildé une image dans un job. Modifiez le pipeline pour qu'un autre job partage/utilise cette image.

2 façons : **1)** pousser l'image vers un registre (mon cas : l'ACR) et la `pull` dans l'autre job ; **2)** `docker save` l'image en `.tar` + la passer en `artifacts` GitLab, puis `docker load` (TD 07 p.12).

## 9. Schéma archi avec les ports (conteneurs + hôte). Changez le mapping : port interne 5050, mappé sur 9090.

Hôte (localhost:80 / 8080 / 8000) → réseau Docker → conteneurs. Pour le changement : `ports: "9090:5050"` (hôte 9090 → conteneur 5050). ⚠ Pour Spring, faut **aussi** lui dire d'écouter sur 5050 ( `SERVER_PORT=5050` ) et mettre à jour l'upstream nginx.

## 10. Le prof veut déployer votre Terraform sur SON compte Azure. Quelles modifs/configs ? Étapes + pourquoi.

Rien à changer dans les `.tf` (tout est en variables). Lui : `az login` (son compte), crée un `terraform.tfvars` avec **ses** noms ( `acr_name` , app names — **uniques globalement** sur Azure, sinon collision), puis `terraform init` + `apply`. Les noms sont externalisés exprès pour ça.

## 11. Exécutez votre Terraform, listez les ressources, montrez leur config, expliquez le rôle de chacune.

`terraform state list` + `az resource list -g rg-abtesting-66045`. **Les 7** : Resource Group (regroupe tout), Managed Identity (identité des apps), ACR (stocke les images), Role Assignment AcrPull (droit de pull sans secret), Service Plan B1 (capacité CPU/RAM), 2 App Services (mes apps Spring + Django).

## 12. Changez le nom de toutes les images avec le suffixe -ESI. Modifs Dockerfiles + pipeline + Terraform. Vérifiez.

**3 fichiers** (PAS les Dockerfiles — le nom est passé au `build`, pas écrit dedans) : `.gitlab-ci.yml` ( `docker build -t ...spring-boot-ESI:latest` ), `terraform/variables.tf` ( `spring_image = "spring-boot-ESI:latest"` ), + le `docker-compose.yml` si je veux nommer en local. Vérif : `az acr repository list` montre les nouveaux noms.

## 13. Supprimez un .dockerignore, reconstruisez l'image, lancez un conteneur, explorez l'arborescence. Montrez ce qui a été copié + pourquoi le .dockerignore est important.

`rm django/.dockerignore` → `docker compose build django` → `docker exec -it app-django sh` → `ls -la /app` : on voit `.git/`, `.idea/`, `db.sqlite3`, `__pycache__`. Importance : **taille** de l'image, **sécurité** (l'historique git peut leak des secrets), **reproductibilité**. Puis `git checkout` pour restaurer.

Forme = TD · Action = consigne · Outil = la techno. Démo locale qui marche + pipelines verts + tags = tes preuves.